

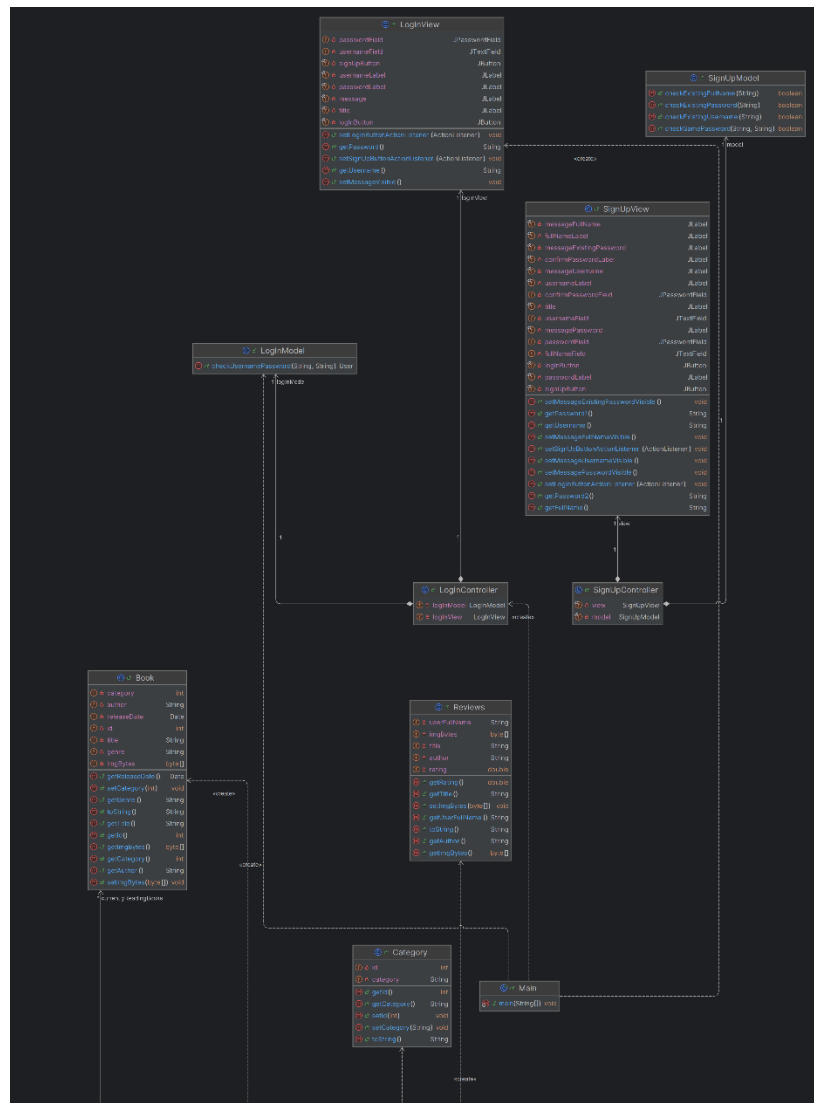
BetterReads - Documentation

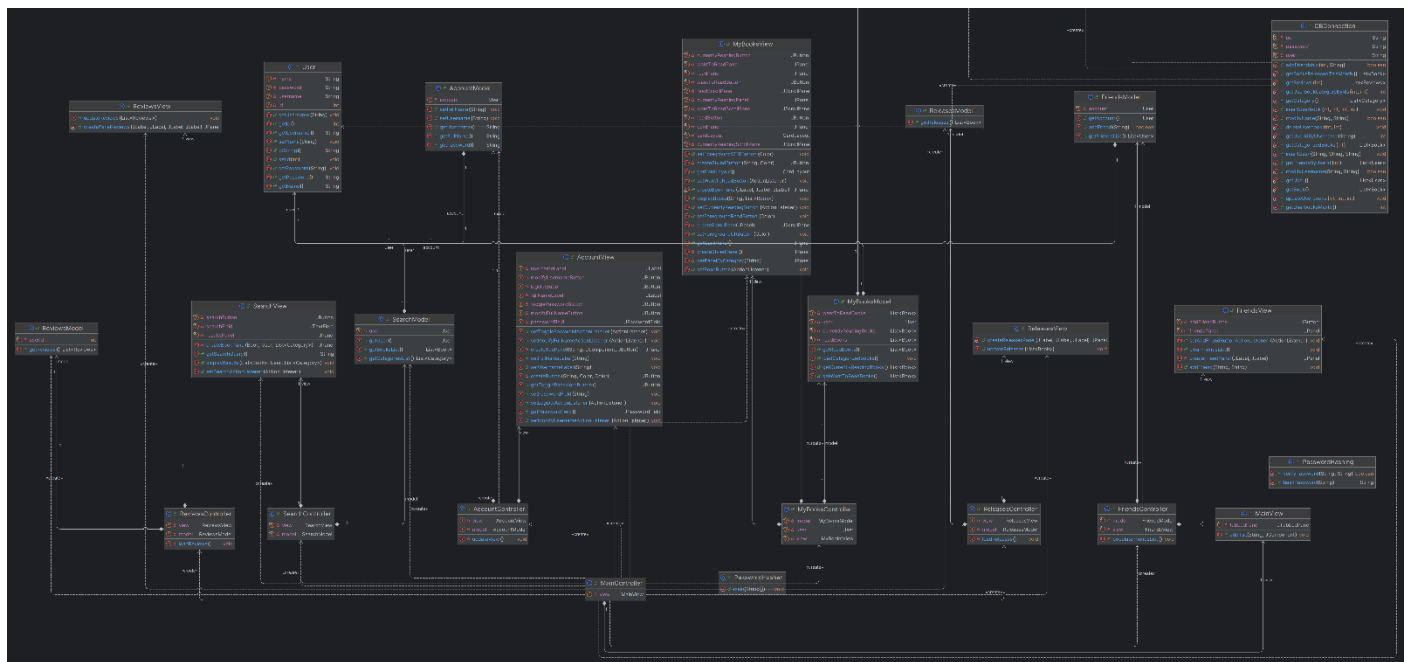
A Java application created by Derşedan Denisa & Florea Deborah

1. Introduction

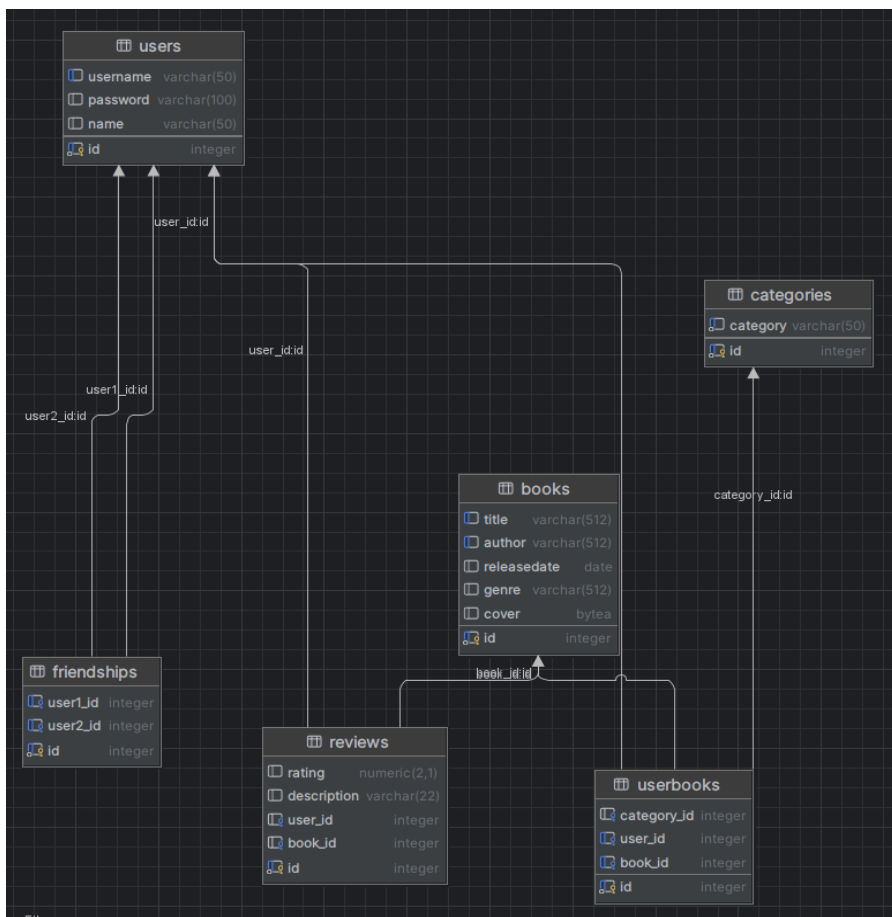
BetterReads is a Java application implemented using the JFrame library. It is designed to make the reading experience less lonely and help you keep track of your own library. The application requires you to create an account and log in to access the available facilities. The main page contains several tabs that allow you to view your friends' reviews (Reviews Tab), this month's releases (Releases Tab), your books in a sorted manner (MyBooks Tab), your account information (Account Tab), your friends (Friends Tab) and search for more books to add to your library (Search Tab).

2. Class Diagram





3. Database

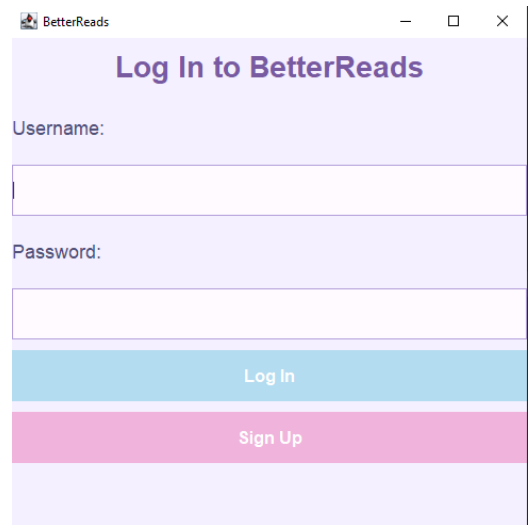


4. Classes Descriptions

The classes below having “MVC” mentioned after their name, signify the fact that a Model-View-Controller type implementation has been used. For ease of explanation, they were taken as a whole structure.

4.1. Log In MVC

The Log In screen appears first, allowing the user to insert their credentials, to access the application’s content or create a new account by clicking the “Sign Up” button. When the user inputs his credentials and clicks “Log In”, a connection to the database is established and the data from the “user” table is iterated to find out if the credentials are valid, otherwise a message appears to warn the user. The data taken as input is used to create a User object, which will be used to display data related to them.



The screenshot shows a web application window titled "BetterReads". The main heading is "Log In to BetterReads". Below the heading, there are two input fields: "Username:" and "Password:". At the bottom of the form, there are two buttons: "Log In" (light blue) and "Sign Up" (pink).

4.2. DBConnection

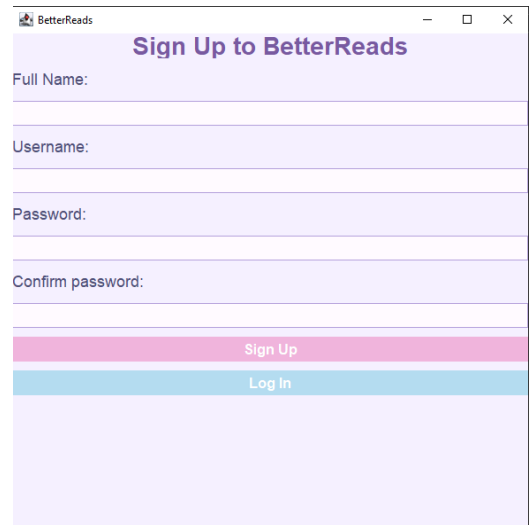
DBConnection is a utility class primarily used by the main classes of the application, such as the pages. It handles data retrieval and updates for the database.

4.3. PasswordHashing

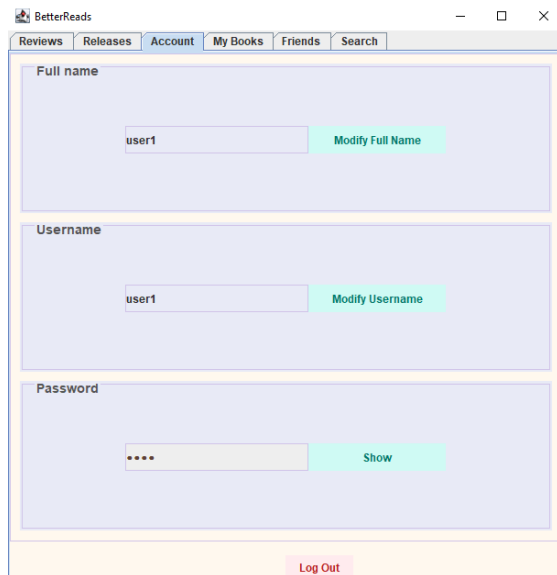
Similar to DBConnection, the PasswordHashing class is a utility class responsible for hashing and verifying passwords. We implemented password hashing using Bcrypt, which streamlined the process significantly. Bcrypt was integrated into our project via Gradle, and we utilized its default functions, such as hashpw for hashing passwords and checkpw for verifying them. When a user logs in, their inputted password is verified using checkpw, and when a new user signs up, their password is hashed using hashpw before being stored in the database.

4.4. Sign Up MVC

The Sign Up page allows the user to set its new data but constrains it to add a unique username in order to be found easier by other users. If the passwords do not match or any other error is encountered, the user receives a message.



4.5. Account MVC



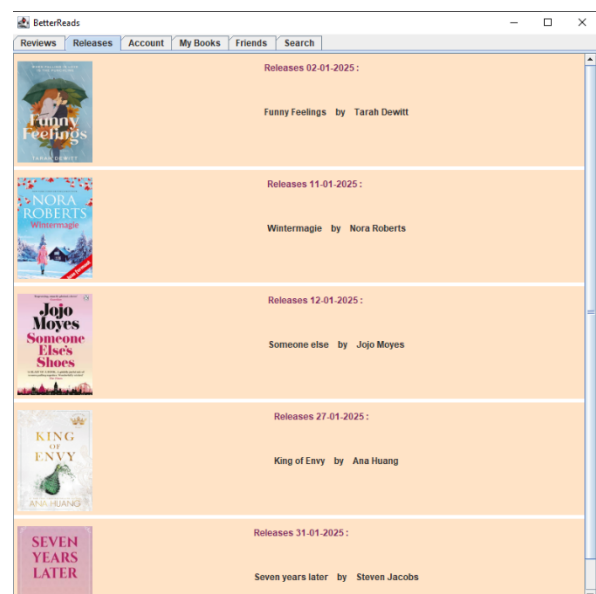
The Account page enables users to view their personal

details, which is its primary purpose. Additionally, users can update their username or full name and view their password. When a "Modify" button is clicked, a confirmation dialog appears, allowing the user to enter a new username or full name. After confirming, the database is updated via the DBConnection class. When the "Show Password" button is clicked, the system retrieves the password from the database, reverses its

hashing, and displays it. The user can log out by clicking the "Log Out" button, which redirects them to the Log In page.

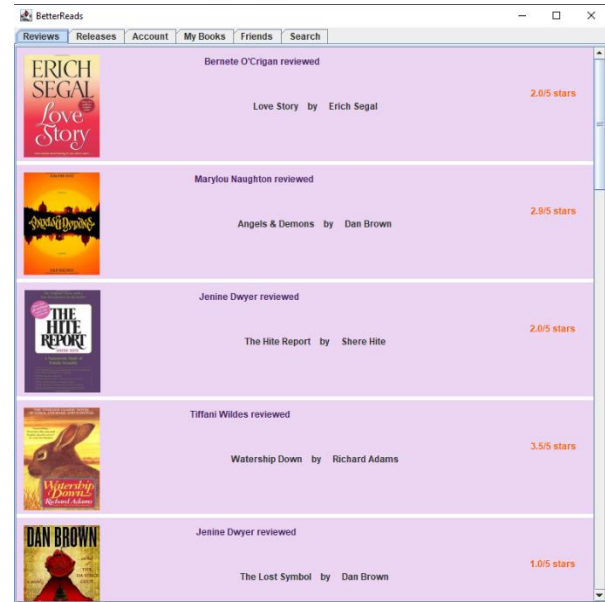
4.6. Releases MVC

The Releases page displays books that have the release date in the current month. Data is retrieved through the DBConnection class from the database and the title, author, release date and cover of each book is displayed in a separate panel. For displaying the panels on all of the pages we used GridBagLayout, since it allowed us the most freedom with arranging the data.

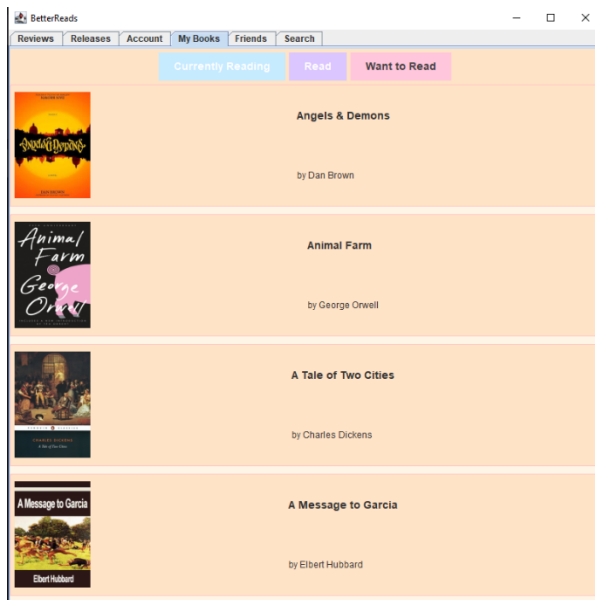


4.7. Reviews MVC

The Reviews tab displays the reviews your friends have left of certain books. Data from the “reviews” joined with “friends” tables is retrieved through the DBConnection class from the database using a join query on 3 different tables and the title, author and cover together with the friend’s username and their rating out of 5 stars are displayed in a separate panel.



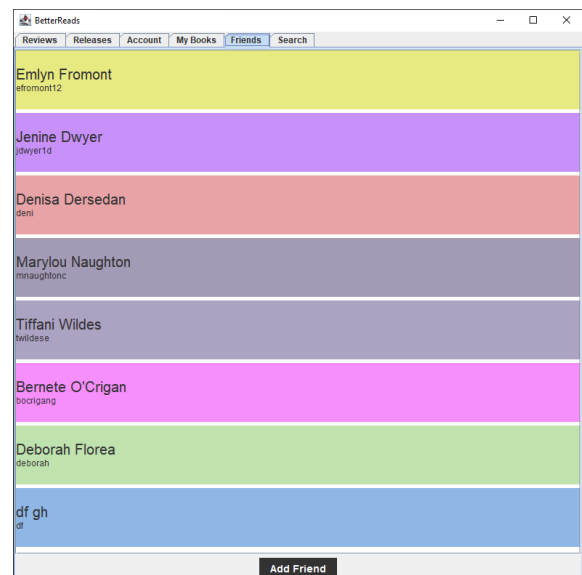
4.8. MyBooks MVC



As the name implies, this page allows users to view books they have categorized as "Currently Reading," "Read," or "Want to Read." Users can choose which category they want to view to see their selected books. A book can be moved from one category to another from the Search page.

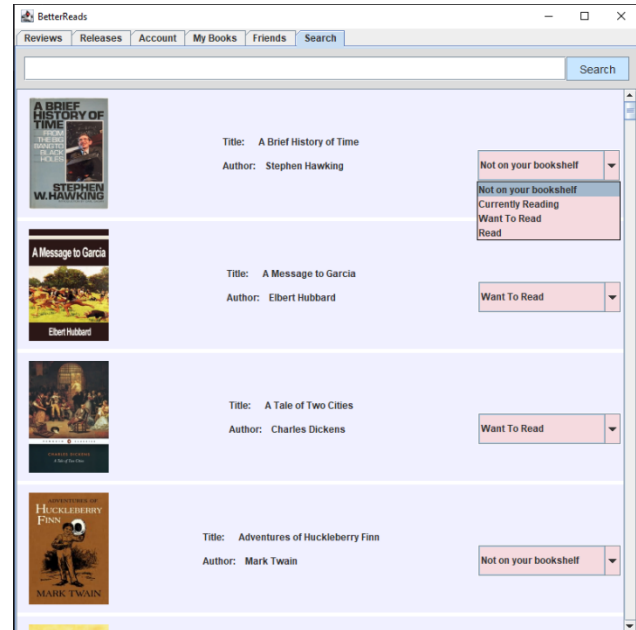
4.9. Friends MVC

The Friends page displays the user's friends and provides the option to add new ones. By clicking the "Add Friend" button, a dialog appears where the user can enter another user's username to establish a friendship. In this application, friendships are bidirectional, meaning only one user needs to initiate the request for the relationship to be created.



4.10. Search MVC

The Search page contains at the top a search bar which searches the given title through the list of books retrieved from the database. Unless a specific title is searched for, we have drawn underneath the list of all books retrieved from the database with title, author, cover and a dropdown that allows the user to select the category it wants a certain book to be in. Changing the category of a book adds/deletes it from the list displayed in the MyBooks tab and changes its state in the “userbooks” table in the database by either updating, deleting or inserting it.



4.11. Database Objects

For some tables from the database, we built specific classes to use the objects of throughout our other tabs.

Book	User	Reviews	Category
<pre> class Book { category: int author: String releaseDate: Date id: int title: String genre: String imgBytes: byte[] getReleaseDate(): Date setCategory(int): void getGenre(): String toString(): String getTitle(): String getId(): int getImgBytes(): byte[] getCategory(): int getAuthor(): String setImgBytes(byte[]): void </pre>	<pre> class User { name: String password: String username: String id: int setUsername(String): void getId(): int getUsername(): String setName(String): void toString(): String setId(int): void setPassword(String): void getPassword(): String getName(): String </pre>	<pre> class Reviews { userFullName: String imgBytes: byte[] title: String author: String rating: double getRating(): double getTitle(): String setImgBytes(byte[]): void getUserFullName(): String toString(): String getAuthor(): String getImgBytes(): byte[] </pre>	<pre> class Category { id: int category: String getId(): int getCategory(): String setId(int): void setCategory(String): void toString(): String </pre>